

CMU · 14-455 | Database System (2019)

CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL

聚合

排序

并发控制

数据库

内存管理

查询规划

分组

树索引

索引构建

查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2: 扫码点击**底部菜单栏**称为 **AI 内容创作者?** 回复 [添砖加瓦]

Lecture #10: Sorting & Aggregation Algorithms

15-445/645 Database Systems (Fall 2019)

<https://15445.courses.cs.cmu.edu/fall2019/>

Carnegie Mellon University

Prof. Andy Pavlo

1 Sorting

We need sorting because in the relation model, tuples in a table have no specific order. Sorting is (potentially) used in ORDER BY, GROUP BY, JOIN, and DISTINCT operators.

We can accelerate sorting using a *clustered* B+tree by scanning the leaf nodes from left to right. This is a bad idea, however, if we use an *unclustered* B+tree to sort because it causes a lot of I/O reads (random access through pointer chasing).

If the data that we need to sort fits in memory, then the DBMS can use a standard sorting algorithm (e.g., quicksort). If the data does not fit, then the DBMS needs to use external sorting that is able to spill to disk as needed and prefers sequential over random I/O.

2 External Merge Sort

Divide-and-conquer sorting algorithm that splits the data set into separate *runs* and then sorts them individually. It can spill runs to disk as needed then read them back in one at a time.

Phase #1 – Sorting: Sort small chunks of data that fit in main memory, and then write back to disk.

Phase #2 – Merge: Combine sorted sub-files into a larger single file.

Two-way Merge Sort

1. Pass #0: Reads every B pages of the table into memory. Sorts them, and writes them back into disk. Each sorted set of pages is called a **run**.
2. Pass #1,2,3,...: Recursively merges pairs of runs into runs twice as long.

Number of Passes: $1 + \lceil \log_2 N \rceil$

Total I/O Cost: $2N \times (\# \text{ of passes})$

General (K -way) Merge Sort

1. Pass #0: Use B buffer pages, produce N/B sorted runs of size B .
2. Pass #1,2,3,...: Recursively merge $B - 1$ runs.

Number of Passes = $1 + \lceil \log_{B-1} \lceil \frac{N}{B} \rceil \rceil$

Total I/O Cost: $2N \times (\# \text{ of passes})$

Double Buffering Optimization

Prefetch the next run in the background and store it in a second buffer while the system is processing the current run. This reduces the wait time for I/O requests at each step by continuously utilizing the disk.

3 Aggregations

An aggregation operator in a query plan collapses the values of one or more tuples into a single scalar value. There are two approaches for implementing an aggregation: (1) sorting and (2) hashing.

Sorting

The DBMS first sorts the tuples on the GROUP BY key(s). It can use either an in-memory sorting algorithm if everything fits in the buffer pool (e.g., quicksort) or the external merge sort algorithm if the size of the data exceeds memory.

The DBMS then performs a sequential scan over the sorted data to compute the aggregation. The output of the operator will be sorted on the keys.

Hashing

Hashing can be computationally cheaper than sorting for computing aggregations. The DBMS populates an ephemeral hash table as it scans the table. For each record, check whether there is already an entry in the hash table and perform the appropriate modification.

If the size of the hash table is too large to fit in memory, then the DBMS has to spill it to disk:

- **Phase #1 – Partition:** Use a hash function h_1 to split tuples into partitions on disk based on target hash key. This will put all tuples that match into the same partition. The DBMS spills partitions to disk via output buffers.
- **Phase #2 – ReHash:** For each partition on disk, read its pages into memory and build an in-memory hash table based on a second hash function h_2 (where $h_1 \neq h_2$). Then go through each bucket of this hash table to bring together matching tuples to compute the aggregation. Note that this assumes that each partition fits in memory.

During the ReHash phase, the DBMS can store pairs of the form (GroupByKey→RunningValue) to compute the aggregation. The contents of RunningValue depends on the aggregation function. To insert a new tuple into the hash table:

- If it finds a matching GroupByKey, then update the RunningValue appropriately.
- Else insert a new (GroupByKey→RunningValue) pair.

CMU · 14-455 | Database System (2019)

CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL

聚合

排序

并发控制

数据库

内存管理

查询规划

分组

树索引

索引构建

查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2：扫码点击**底部菜单栏**称为 **AI 内容创作者**？回复 [添砖加瓦]